EXPERIMENT 7

Shravanya Andhale
D15A-30

AIM: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

THEORY:

1. Dataset Source

- Dataset: CEEW India Residential Energy Survey Microdata
- Source: Council on Energy, Environment and Water
- Accessed through Government of India microdata repository

Source link:
https://www.kaggle.com/code/raitest/india-residential-energy-survey-ires-2020-eda/input?select=CEEW+-+IRES+Data.csv

2. Dataset Description

Target Variable:

- q609_prepaid_meter_int

Data Cleaning:

- Retained only valid responses 0 and 1
- Removed values coded as 99 and missing observations

Binary Target Created:

- Prepaid_Interest = 1 if household is interested in prepaid electricity meters
- Prepaid_Interest = 0 if household is not interested

Test Set Distribution:

- Class 0 (Not Interested): 1137
- Class 1 (Interested): 283

Predictor Variables Used:

- asset_index_1
- q208_priminc_earner_edu
- q202_resp_age
- q213_no_members
- q302_grid_hrs_no
- q308_grid_voltage_low_app
- q326_satis_electricity
- q314_a_online_pay_ever_yn
- q401_bee_star_label_heard_yn
- q232_mobile_smart_n

These variables represent socio-economic status, infrastructure reliability, electricity experience, and digital engagement.

3. Mathematical Formulation of Artificial Neural Network

An Artificial Neural Network consists of interconnected neurons arranged in layers.

Each neuron performs the operation:

$z = w_1x_1 + w_2x_2 + ... + w_{\square}x_{\square} + b$

The output is passed through an activation function.

For hidden layers:

ReLU Activation:
$f(x) = \max(0, x)$

For binary classification output:

Sigmoid Activation:
$\sigma(x) = 1 / (1 + e^{-x})$

Loss Function Used:
Binary Cross Entropy

$\text{Loss} = -[y \log(p) + (1 - y) \log(1 - p)]$

Optimization Algorithm:
Adam optimizer was used to update network weights using gradient descent.

Network Architecture:

- Input Layer: 10 features
- Hidden Layer 1: 64 neurons (ReLU)
- Hidden Layer 2: 32 neurons (ReLU)
- Hidden Layer 3: 16 neurons (ReLU)
- Dropout layers applied for regularization
- Output Layer: 1 neuron with sigmoid activation

4. Algorithm Limitations

- Requires large datasets for optimal performance
- Model interpretability is lower compared to traditional algorithms
- Sensitive to class imbalance
- Training time higher compared to simpler models

- Hyperparameter tuning required for architecture optimization

5. Methodology / Workflow

- Import required libraries
- Load dataset
- Clean target variable and remove invalid responses
- Create binary classification label
- Select relevant socio-economic and behavioural predictors
- Remove missing values
- Perform stratified train-test split
- Apply StandardScaler for feature normalization
- Construct Artificial Neural Network architecture using Keras
- Train ANN using training dataset
- Monitor training and validation loss and accuracy
- Generate predictions on test dataset
- Evaluate model using accuracy, confusion matrix, classification report, and ROC-AUC
- Visualize performance using ROC curve and confusion matrix heatmap

6. Performance Analysis

ANN Model Results:

- **Test Accuracy: 0.80**
- **ROC-AUC: 0.6387**

Confusion Matrix:

[[1128 9]
[276 7]]

Classification Insights:

- Recall for households interested in prepaid meters: 0.02
- Precision for interested households: 0.44
- Majority class (not interested) predicted accurately
- Model struggles to identify minority class cases

Interpretation:

The ANN model successfully captures overall patterns in the dataset, achieving an accuracy of approximately 80 percent. However, due to class imbalance, the model primarily predicts the dominant class. This results in very low recall for households interested in prepaid meters.

The ROC-AUC value of 0.6387 indicates moderate discrimination ability. This suggests that socio-economic characteristics, electricity experience, and digital awareness partially explain prepaid meter interest but do not fully determine it.

7. Hyperparameter Tuning

Model parameters used:

- Optimizer: Adam
- Loss Function: Binary Cross Entropy
- Epochs: 30
- Batch Size: 32
- Activation Functions: ReLU for hidden layers and Sigmoid for output layer
- Dropout layers applied to reduce overfitting

Hyperparameter tuning involved adjusting network depth and dropout levels to improve generalization performance.

OUTPUT

- ANN successfully trained on the household electricity dataset.
- Model achieved an overall accuracy of 0.80 on the test dataset.
- ROC-AUC value of 0.6387 indicates moderate classification capability.
- Confusion matrix shows strong prediction of majority class but limited detection of minority class.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
df = pd.read_csv("CEEW - IRES Data.csv", low_memory=False)

print("Dataset shape:", df.shape)
```

```
Dataset shape: (14851, 517)
```

```
df_ann = df[df['q609_prepaid_meter_int'].isin([0,1])]

df_ann['Prepaid_Interest'] = np.where(
    df_ann['q609_prepaid_meter_int'] == 1, 1, 0
)

print(df_ann['Prepaid_Interest'].value_counts())
```

```
Prepaid_Interest
0    9533
1    2069
Name: count, dtype: int64
/tmp/ipykernel_3631/1817887926.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_ann['Prepaid_Interest'] = np.where(
```

```python
features = [
'asset_index_1',
'q208_priminc_earner_edu',
'q202_resp_age',
'q213_no_members',
'q302_grid_hrs_no',
'q308_grid_voltage_low_app',
'q326_satis_electricity',
'q314_a_online_pay_ever_yn',
'q401_bee_star_label_heard_yn',
'q232_mobile_smart_n'
]

df_model = df_ann[features + ['Prepaid_Interest']].dropna()

print("Final dataset shape:", df_model.shape)
```

```
Final dataset shape: (7100, 11)
```

```python
X = df_model[features]
y = df_model['Prepaid_Interest']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```python
scaler = StandardScaler()

X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

```python
model = Sequential()

model.add(Dense(64, activation='relu', input_shape=(X_train_s.shape[1],)))
model.add(Dropout(0.3))

model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(16, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

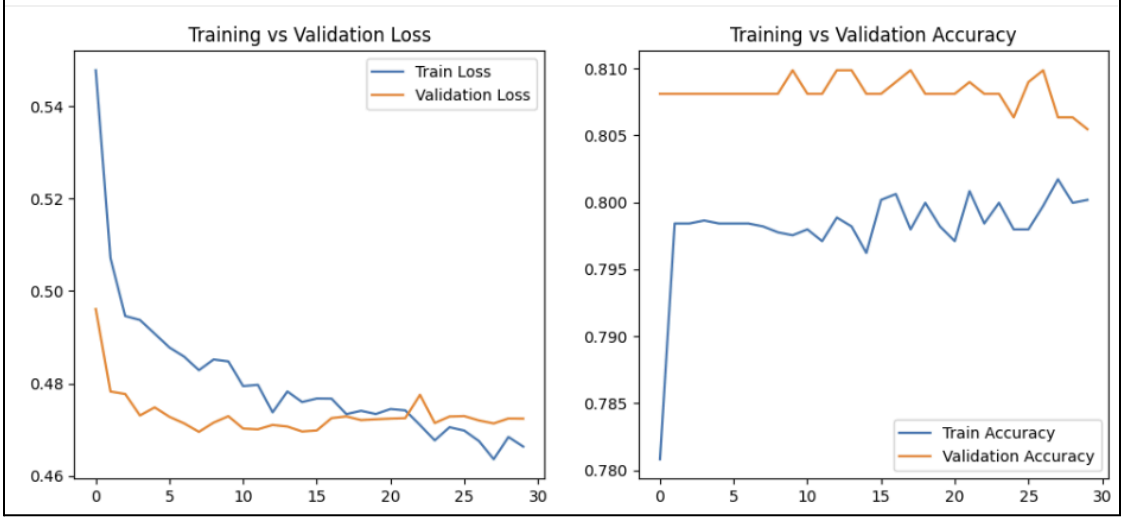
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	704
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

```
Total params: 3,329 (13.00 KB)
Trainable params: 3,329 (13.00 KB)
Non-trainable params: 0 (0.00 B)
```

```python
history = model.fit(
    X_train_s,
    y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=32,
    verbose=1
)
```

```
Epoch 2/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7984 - loss: 0.5071 - val_accuracy: 0.8081 - val_loss: 0.4782
Epoch 3/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7984 - loss: 0.4946 - val_accuracy: 0.8081 - val_loss: 0.4777
Epoch 4/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7986 - loss: 0.4937 - val_accuracy: 0.8081 - val_loss: 0.4731
Epoch 5/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7984 - loss: 0.4908 - val_accuracy: 0.8081 - val_loss: 0.4748
Epoch 6/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7984 - loss: 0.4878 - val_accuracy: 0.8081 - val_loss: 0.4728
Epoch 7/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7984 - loss: 0.4858 - val_accuracy: 0.8081 - val_loss: 0.4714
Epoch 8/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7982 - loss: 0.4829 - val_accuracy: 0.8081 - val_loss: 0.4695
Epoch 9/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7978 - loss: 0.4852 - val_accuracy: 0.8081 - val_loss: 0.4715
Epoch 10/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7975 - loss: 0.4848 - val_accuracy: 0.8099 - val_loss: 0.4729
Epoch 11/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7980 - loss: 0.4794 - val_accuracy: 0.8081 - val_loss: 0.4702
Epoch 12/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7971 - loss: 0.4797 - val_accuracy: 0.8081 - val_loss: 0.4701
Epoch 13/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7989 - loss: 0.4737 - val_accuracy: 0.8099 - val_loss: 0.4710
Epoch 14/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7982 - loss: 0.4783 - val_accuracy: 0.8099 - val_loss: 0.4707
Epoch 15/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7962 - loss: 0.4760 - val_accuracy: 0.8081 - val_loss: 0.4696
Epoch 16/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8002 - loss: 0.4767 - val_accuracy: 0.8081 - val_loss: 0.4698
Epoch 17/30
142/142 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8006 - loss: 0.4767 - val_accuracy: 0.8090 - val_loss: 0.4725
Epoch 18/30
```

```python
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Training vs Validation Loss")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Training vs Validation Accuracy")
plt.legend()

plt.show()
```



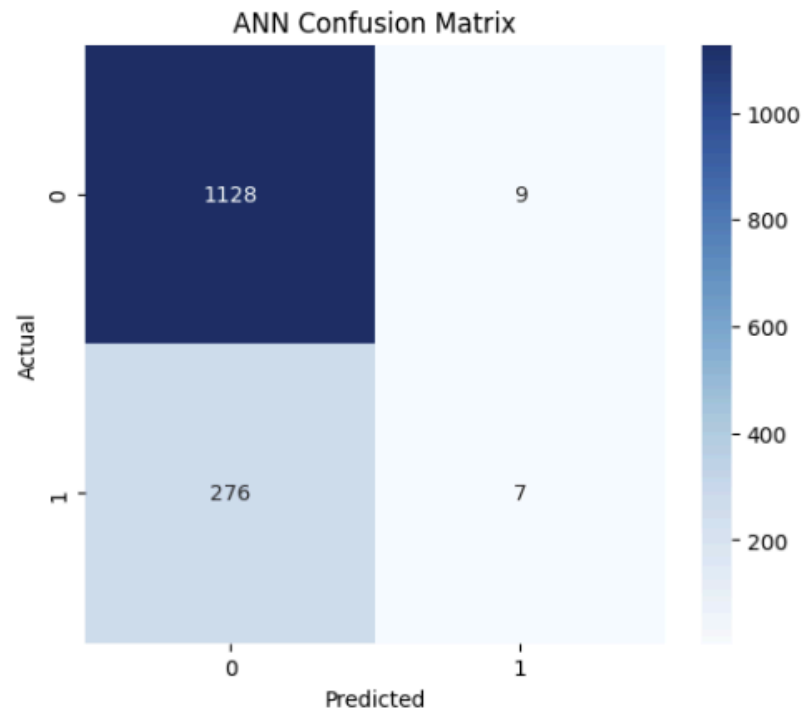
```python
y_prob = model.predict(X_test_s)
y_pred = (y_prob > 0.5).astype(int)
```

```
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

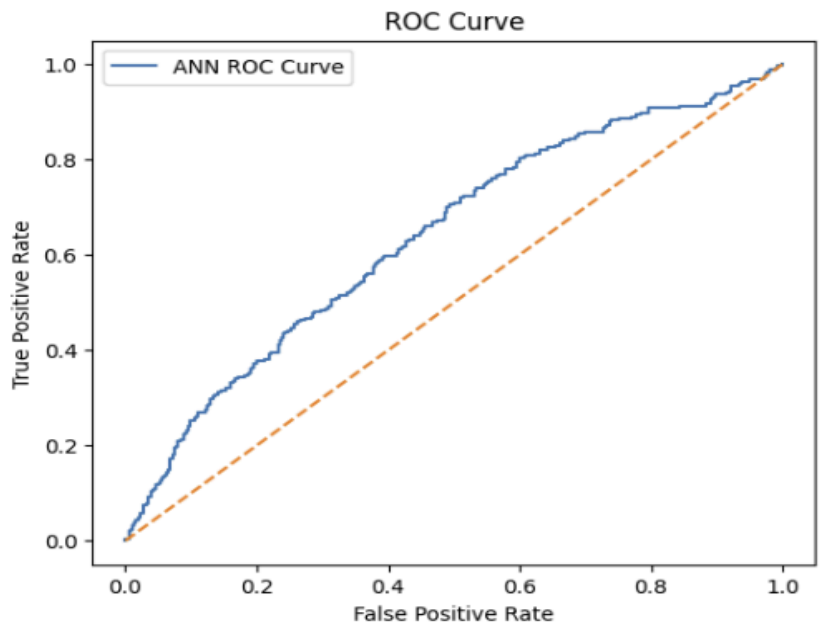
```python
print("Test Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("ROC AUC:", roc_auc_score(y_test, y_prob))
```

```
Test Accuracy: 0.7992957746478874

Confusion Matrix:
[[1128    9]
 [ 276    7]]

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.99      0.89      1137
           1       0.44      0.02      0.05       283

    accuracy                           0.80      1420
   macro avg       0.62      0.51      0.47      1420
weighted avg       0.73      0.80      0.72      1420

ROC AUC: 0.6386840330545637
```

```
plt.figure(figsize=(6,5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("ANN Confusion Matrix")

plt.show()
```



```
fpr, tpr, _ = roc_curve(y_test, y_prob)

plt.figure(figsize=(6,5))

plt.plot(fpr, tpr, label="ANN ROC Curve")
plt.plot([0,1],[0,1], linestyle='--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")

plt.legend()

plt.show()
```



CONCLUSION

- Artificial Neural Networks can be applied effectively to household electricity datasets containing socio-economic and behavioural variables.
- The model demonstrates moderate predictive ability for prepaid meter interest.
- Class imbalance significantly affects minority class prediction performance.
- Structural and behavioural variables provide partial insight into prepaid meter adoption interest but cannot fully explain it.
- ANN models provide a flexible framework capable of capturing nonlinear relationships between household characteristics and energy behaviour.